

Chaining Tasks by Using Continuation Tasks in C#

In this article, I am going to discuss **Chaining Tasks by Using Continuation Tasks in C#** with Examples. Please read our previous article where we discussed [Task-Based Asynchronous Programming in C#](#) with Examples.

While working with asynchronous programming, it is very common to invoke one asynchronous operation from another asynchronous operation passing the data once it completes its execution. This is called continuations and in the traditional approach, this has been done by using the callback method which is a little difficult to understand.

But with the introduction of Task Parallel Library (TPL), the same functionality can be achieved very easily by using continuation tasks. In simple words, we can define a continuation task as an asynchronous task that is going to be invoked by another task (i.e. known as the antecedent).

Creating a continuation for a single antecedent

In C#, you can create a continuation by calling the `ContinueWith` method that is going to execute when its antecedent has completed its execution.

In the following example, the antecedent task i.e. `task1` returns an integer value. When it completes its executions, then it passes that value to the continuation task and that continuation task does some operations and returns the result as a string.

```
using System;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Task<string> task1 = Task.Run(() =>
            {
                return 12;
            }).ContinueWith((antecedent) =>
            {
                return $"The Square of {antecedent.Result} is: {antecedent.Result * antecedent.Result}";
            });
            Console.WriteLine(task1.Result);

            Console.ReadKey();
        }
    }
}
```

Output: **The Square of 12 is : 144**

Scheduling Different Continuation Tasks

The `ContinueWith` method has some overloaded versions that you can use to configure with multiple options when the continuation will run. In this way, you can add different continuation methods that will run when an exception

occurred, when the Task is canceled, or the Task is completed successfully. Let us see an example to understand this.

```
using System;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Task<int> task = Task.Run(() =>
            {
                return 10;
            });
            task.ContinueWith((i) =>
            {
                Console.WriteLine("Task Canceled");
            }, TaskContinuationOptions.OnlyOnCanceled);
            task.ContinueWith((i) =>
            {
                Console.WriteLine("Task Faulted");
            }, TaskContinuationOptions.OnlyOnFaulted);
            var completedTask = task.ContinueWith((i) =>
            {
                Console.WriteLine("Task Completed");
            }, TaskContinuationOptions.OnlyOnRanToCompletion);
            completedTask.Wait();
            Console.ReadKey();
        }
    }
}
```

In the next article, I am going to discuss [How to Attach Child Tasks to a Parent Task in C#](#) with Examples. In this article, I try to explain **how to Chaining Tasks by Using Continuation Tasks in C#** with some examples. I hope you enjoy this article.